Day 36: Function Returning None

Day 36 — Function Returning None

1. Day Number

Day 36

2. Topic Name

`None` and Invalid Result

Today you will learn how a function can say:

> "I don't have a valid result to return."

Python commonly represents this using:

```
None
```

---

3. Connection

Earlier, you used functions that returned useful values.

For example:

```
function
   ↓
calculation
   ↓
return result
```

Maybe a function calculates a price:

```
return total
```

But what happens when the input itself is invalid?

For example:

```
amount = -100
```

A negative amount may not make sense for your program.

Instead of returning a normal amount, the function can return:

```
None
```

So today's idea is:

```
Valid input
    ↓
return useful value

Invalid input
```

```
    ↓
return None
```

---

4. Important Topics

Today's important concepts are:

- function
- `return`
- `None`
- condition
- `if`
- validating input

The main pattern is:

```
def function_name(value):

    if some_condition:
        return value

    return None
```

The important thing is understanding the **idea**, not memorizing the pattern.

---

5. Foundational Notes

What is None?

`None` is a special value in Python.

It represents:

> no value / no valid result / nothing available

For example:

```
result = None
```

This does **not** mean:

```
result = 0
```

And it does not mean:

```
result = ""
```

They are different values.

Think of them like this:

```
0       → a number whose value is zero
""      → an empty string
[]      → an empty list
None    → no value / no result
```

Why do functions return None?

Suppose a function expects a positive number.

Valid:

```
100
50
1
```

Invalid:

```
0
-10
-500
```

Instead of pretending that an invalid value is correct, the function can return:

```
None
```

Example idea:

```
Input: 100
      ↓
Is 100 positive?
      ↓
Yes
      ↓
Return 100
```

But:

```
Input: -100
      ↓
Is -100 positive?
      ↓
No
      ↓
Return None
```

None is an actual Python value

You can store it in a variable:

```
result = None
```

You can also check for it.

The common Python style is:

```
if result is None:
```

rather than:

```
if result == None:
```

For now, remember:

```
is None
```

is the preferred way to check whether something is `None`.

---

return stops the function

Remember that when Python reaches `return`, the function finishes.

Conceptually:

```
function starts
     ↓
check condition
     ↓
return something
     ↓
function ends
```

Any code after that particular executed `return` will not run.

Explicit and implicit None

There are two ways a function may produce `None`.

Explicit

You deliberately write:

```
return None
```

Implicit

If a function reaches the end without any `return`, Python automatically returns `None`.

Example:

```
def show_message():
    print("Hello")
```

This prints something, but it doesn't explicitly return a value.

Therefore its return value is `None`.

For today's problem, explicitly using:

```
return None
```

is useful because it makes your intention clear.

---

6. Easy Example

Consider a different problem.

Create a function that accepts an age.

If the age is greater than `0`, return the age.

Otherwise return `None`.

Example structure:

```
def check_age(age):
    if age > 0:
        return age

    return None
```

Then:

```
result = check_age(25)
```

The result would be:

```
25
```

But:

```
result = check_age(-5)
```

would give:

```
None
```

The important pattern is:

```
receive input
     ↓
check validity
     ↓
valid? ── Yes ──→ return value
  │
  No
  ↓
return None
```

---

7. Problem Statement

Create a function that takes an `amount`.

If the amount is **positive**, return the amount.

Otherwise return:

```
None
```

Examples:

```
amount = 200
Result = 200
```

```
amount = 0
Result = None
```

```
amount = -50
Result = None
```

Do not worry about user input or complicated validation yet.

Focus on:

```
function
+
condition
+
return
+
None
```

---

8. Concepts Used

Function

You need a function that receives one value:

```
amount
```

Conceptually:

```
def function_name(amount):
```

Condition

You need to determine whether the amount is valid.

Ask:

```
Is amount greater than 0?
```

The condition will therefore involve:

```
amount > 0
```

Return value

When the amount is valid:

```
return the amount
```

None

When the amount is invalid:

```
return None
```

So the overall flow is:

```
amount
  ↓
function
  ↓
amount > 0 ?
  ├── Yes → return amount
  │
  └── No  → return None
```

---

9. Thought Process

Before writing Python, think about the problem step by step.

Suppose:

```
amount = 500
```

Ask:

```
Is 500 > 0?
```

Yes.

Therefore:

```
return 500
```

Now suppose:

```
amount = -100
```

Ask:

```
Is -100 > 0?
```

No.

Therefore:

```
return None
```

Now consider:

```
amount = 0
```

The condition is:

```
0 > 0
```

That is false.

Therefore:

```
return None
```

This is an important boundary case.

Your logic is therefore simply:

```
Receive amount
       ↓
Check amount > 0
       ↓
       |
   ┌───┴────┐
 Yes        No
  ↓          ↓
amount      None
```

---

10. Pseudocode

Write the logic without Python syntax first:

```
FUNCTION validate_amount(amount)

    IF amount is greater than 0
        RETURN amount

    OTHERWISE
        RETURN None

END FUNCTION
```

Then use the function:

```
result = call function with an amount

print result
```

Notice the function has **two possible results**:

```
positive amount
or
None
```

---

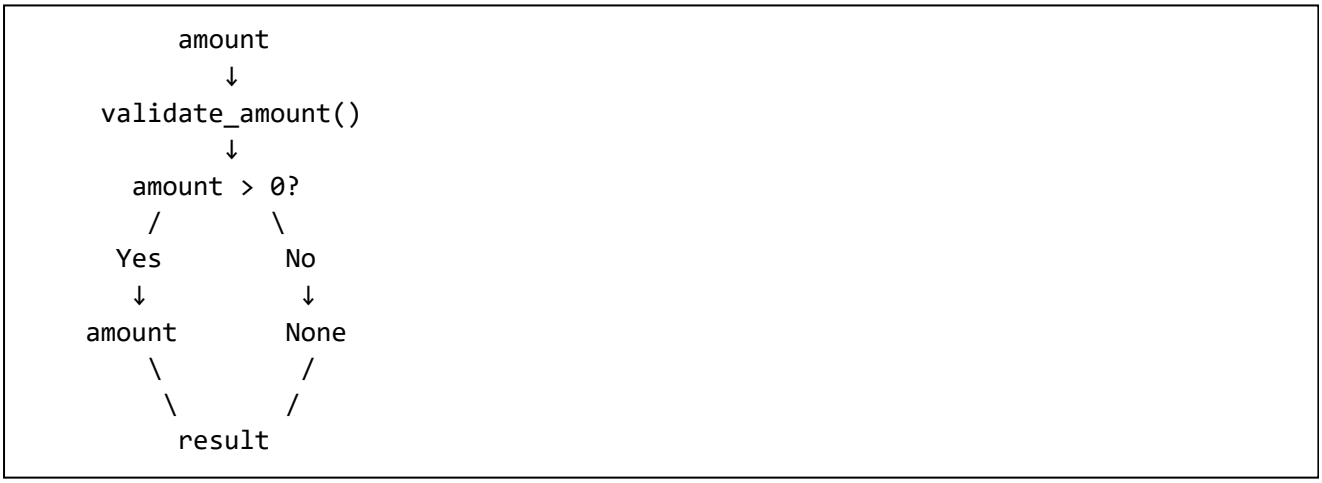
11. Suggested Solving Approach — Functional Approach

Use a small function whose only responsibility is to validate and return the amount.

Think of it as:

```
Input
  ↓
Function
  ↓
Validation
  ↓
Output
```

More specifically:



Keep the function simple.

It should not need to:

- create dictionaries
- create lists
- use loops
- calculate discounts
- print receipts

Today you are isolating one important concept:

> A function can return `None` when it cannot produce a valid result.

---

12. Easy Edge Cases

Edge Case 1 — Amount is 0

Input:

```
0
```

Ask:

```
Is 0 > 0?
```

No.

Expected result:

```
None
```

Edge Case 2 — Negative Amount

Input:

```
-100
```

Ask:

```
Is -100 > 0?
```

No.

Expected result:

```
None
```

Edge Case 3 — Small Positive Amount

Input:

```
1
```

Ask:

```
Is 1 > 0?
```

Yes.

Expected result:

```
1
```

This helps you verify that the boundary between valid and invalid values is correct.

```
-1 → None
 0 → None
 1 → 1
```

---

13. Expected Input

Your function takes one numeric amount.

Example:

```
100
```

Other test inputs could be:

```
500
1
0
-20
```

You do not need `input()` unless you want extra practice.

You can pass the value directly to the function.

---

14. Expected Output

For:

```
amount = 100
```

Expected result:

```
100
```

For:

```
amount = 0
```

Expected result:

```
None
```

For:

```
amount = -50
```

Expected result:

```
None
```

So:

```
Input      Output

100   →    100
20    →    20
1     →    1
0     →    None
-1    →    None
-100  →    None
```

---

15. Hint Only

Start by creating a function with one parameter:

```
amount
```

Inside the function, ask yourself:

```
Is amount greater than 0?
```

If yes:

```
return the amount
```

Otherwise:

```
return None
```

Think in this form:

```
def __________(amount):

    if __________:
        return __________

    return __________
```

Then call the function and store its returned value:

```
result = __________
```

Finally, print `result`.

Key lesson for Day 36:

```
Valid input   → return useful value
Invalid input → return None
```

`None` gives your function a clean way of saying:

> **“There is no valid result for this input.”**